Proyecto Final

Redes Neuronales

Escuela de Ingeniería de Sistemas y Computación

Oscar David Cuaical Lopez

Diciembre 2025

1. Resumen

El presente proyecto aborda el problema de clasificación de texto con múltiples etiquetas para el análisis de sentimientos (positivo, neutro y negativo) en el dataset Twitter US Airline Sentiment. Debido a que el dataset presenta un fuerte desbalance con mayoría de comentarios negativos, el trabajo se enfoca en comparar arquitecturas que logren aprender correctamente todas las categorías, evitando que el modelo favorezca únicamente a la clase dominante.

El desarrollo se llevó a cabo en dos fases (entregas). En la primera fase, se implementó un modelo de Perceptrón Multicapa (MLP) utilizando una representación vectorial basada en frecuencia (TF-IDF) con reducción de dimensionalidad mediante TruncatedSVD. En la segunda fase, se exploraron modelos secuenciales profundos, específicamente una Red Neuronal Recurrente (Simple RNN) y una Long Short-Term Memory (LSTM), utilizando Word Embeddings entrenables para capturar relaciones semánticas y el orden secuencial del texto.

Los resultados demuestran que, si bien el MLP generaliza bien la estructura de los datos, las arquitecturas con memoria (LSTM) ofrecen una mayor robustez en la detección de clases minoritarias (neutro y positivo).

2. Introducción

El objetivo fundamental de este informe es analizar y comparar el desempeño de diferentes arquitecturas de redes neuronales aplicadas a la tarea de análisis de sentimientos en un entorno de datos reales y desbalanceados. El problema abordado consiste en predecir la polaridad de tweets relacionados con aerolíneas estadounidenses, clasificándolos en sentimientos positivos, neutros o negativos. Para tal fin, el proyecto se basa dos etapas de desarrollo que permiten observar la evolución de los resultados con los diferentes modelos implementados.

En la primera entrega, se llevó a cabo la implementación de una red neuronal Perceptrón Multicapa (MLP). Este enfoque se basó en un preprocesamiento basado en la técnica TF-IDF para la vectorización del texto, complementado con TruncatedSVD para la reducción de dimensionalidad, buscando eficiencia computacional y la validación de características estadísticas. Sin embargo, este tipo de red no es capaz de capturar los patrones complejos en los tweets, pues, aunque ofreció buenos resultados aún se mostraba la tendencia de inclinarse hacia la clase dominante, que en este caso fue la negativa, por este motivo en la segunda entrega se desarrollaron modelos capaces de dar mayor confiabilidad a las predicciones.

En esta segunda entrega, se implementaron y compararon una Red Neuronal Recurrente (RNN) simple y una red LSTM, ambas alimentadas por capas de Embeddings entrenables que transforman los índices de palabras en vectores densos, preservando la estructura secuencial de los mensajes.

Uno de los componentes más difíciles de manejar fue el desbalance de clases, dado que la clase negativa supera por mucho a las demás. Para solucionar este problema y evitar el sobreajuste, se integraron técnicas de regularización como Dropout y penalización L2, junto con el uso de pesos de clase (class weights) durante el entrenamiento y la implementación de Early Stopping. A través de este documento se detalla la configuración metodológica, la selección de hiperparámetros y el análisis de las métricas de evaluación resultantes, permitiendo determinar qué arquitectura ofrece el balance óptimo entre precisión global y capacidad de recuperación de las clases minoritarias.

3. Metodología de desarrollo

En esta sección se explican los pasos que se siguieron para construir los modelos de predicción, empezando por la limpieza de los datos hasta llegar a la implementación. Como el proyecto se desarrolló en dos entregas diferentes, aquí se detalla la forma de procesamiento de los datos entre una y otra. Inicialmente, se usó un enfoque basado en contar la frecuencia de las palabras, y para esta segunda entrega se pasó a una representación numérica que respeta el orden de la secuencia ajustándose a lo que necesitan las redes neuronales recurrentes.

3.1. Preprocesamiento de datos

En primer lugar, se realizó la carga del dataset, para tener un mejor acceso a estos datos se descargó el dataset de Kaggle y posteriormente se cargó este mismo dataset a un repositorio de GitHub para usarlo directamente en el código. Como segundo paso se aplicó una limpieza general de los datos para eliminar términos innecesarios y tener una mejor estructura de los tweets. El proceso consistió en pasar todo el texto a minúsculas y usar expresiones regulares para eliminar elementos que no ayudan a identificar el sentimiento, como las menciones a usuarios, los enlaces web y los caracteres especiales o números. También se quitaron los espacios sobrantes para dejar el texto listo y uniforme.

Para la primera entrega, donde se trabajó con un Perceptrón Multicapa (MLP), se eligió una forma de procesar el texto que no tiene en cuenta el orden de las palabras, sino cuántas veces aparecen. Se utilizó la técnica TF-IDF para convertir los tweets en vectores numéricos según la importancia de cada palabra. Sin embargo, este método generaba una cantidad de datos muy grande, para solucionar esto se aplicó una reducción de dimensionalidad con TruncatedSVD para comprimir la información y hacerla más manejable para la red neuronal simple, quedándose con las características más importantes sin que el procesamiento fuera tan pesado.

En cambio, para la segunda entrega, se usaron redes RNN y LSTM que sí necesitan entender la secuencia de la frase. Por eso se cambió el método y se usó la tokenización. Se seleccionaron las 5,000 palabras más comunes del vocabulario y se les asignó un número único a cada una. Como las redes neuronales necesitan que todas las entradas tengan el

mismo tamaño, se aplicó un proceso de padding para que todos los tweets tuvieran una longitud fija de 50 números. Si el tweet era más corto se rellenaba con ceros, y si era más largo se recortaba, creando así la matriz necesaria para la capa de Embedding.

Por último, se realizó la división de los datos para el entrenamiento. Se separó el conjunto en un 70% para entrenar, 15% para validar y 15% para probar. También se usó una división estratificada para que la proporción de sentimientos negativos, neutros y positivos fuera la misma en los tres grupos. Además, para solucionar el problema de que hay muchos más comentarios negativos que de los otros tipos, se calcularon los pesos de cada clase antes de entrenar. Esto sirve para indicarle al modelo que debe prestar más atención a las clases con menos datos (neutros y positivos) y así evitar que las ignore.

3.2. Definición de arquitecturas de red neuronal

Para la primera entrega, se diseñó una arquitectura de red neuronal tipo Perceptrón Multicapa (MLP). Un perceptrón multicapa es un tipo de red neuronal feedforward que consta de una capa de entrada, una o más capas ocultas y una capa de salida, y utiliza funciones de activación no lineales para aprender relaciones complejas (IBM, 2025). Dado que la entrada provenía de una reducción de dimensionalidad SVD, la capa inicial se configuró para recibir vectores de 300 características numéricas. La red se estructuró con dos capas densas ocultas: la primera con 32 neuronas y activación ReLU, permitiendo modelar relaciones no lineales, y la segunda con 16 neuronas, también con ReLU. Aunque se probaron múltiples configuraciones, esta fue la que ofreció el mejor rendimiento.

Para evitar sobreajuste en el MLP, se aplicaron técnicas de regularización en cada capa. Se empleó una regularización L2 de 0.0001, que penaliza pesos elevados para evitar que la red memorice ruido del entrenamiento. Asimismo, se añadieron capas de Dropout con una tasa del 50% (0.5); este mecanismo apaga aleatoriamente la mitad de las neuronas durante el entrenamiento, lo que obliga al modelo a aprender patrones más robustos y no depender de neuronas individuales. La capa de salida incluye 3 neuronas con activación Softmax, produciendo probabilidades para las clases negativo, neutro y positivo.

Para la segunda entrega, se utilizó dos tipos de redes neuronales recurrentes RNN Simple y LSTM. Las redes neuronales recurrentes están diseñadas para procesar secuencias

manteniendo un estado interno que actúa como memoria de pasos anteriores, dicho estado permite que la red tome decisiones considerando el contexto previo (MongoDB, s.f.). Durante la configuracion de las redes la primera capa común para ambos modelos (RNN y LSTM) es una capa Embedding, configurada para transformar un vocabulario de 5,000 palabras en vectores densos de 100 dimensiones entrenables. A diferencia del enfoque TF-IDF previo, esta representación permite que el modelo aprenda relaciones semánticas entre palabras. Posteriormente, se añadió un SpatialDropout1D en los dos modelos con tasa de 0.5. A diferencia del Dropout tradicional que elimina neuronas individuales, esta variante elimina dimensiones completas de los vectores, lo cual es más eficaz en tareas de secuencia.

El primer modelo RNN Simple se configuró con una capa recurrente de 32 unidades que procesa secuencias de 50 tokens paso a paso. Se aplicó un Dropout interno y recurrente de 0.3 para controlar el sobreajuste sin afectar en exceso la capacidad de aprendizaje. Debido a que las RNN simples presentan el problema del desvanecimiento del gradiente, tienen dificultades para memorizar dependencias largas, limitándose a patrones cercanos en la secuencia. Para estabilizar el entrenamiento, se usó el optimizador Adam con tasa de aprendizaje de 0.0001.

El segundo modelo utilizado fue una Long Short-Term Memory (LSTM). Una LSTM incorpora compuertas que controlan el flujo de información para decidir qué datos mantener o descartar a lo largo del tiempo (MathWorks, 2024). Además, las LSTM están diseñadas específicamente para aprender dependencias a largo plazo mediante una celda de memoria que mitiga el problema del gradiente que se desvanece (DataCamp, 2024).Este modelo también se configuró con 32 unidades, pero con una estructura interna más compleja que la SimpleRNN. Dado que las LSTM suelen memorizar con facilidad (lo que incrementa el riesgo de sobreajuste), se aplicó una regularización más estricta: penalización L2 de 0.001 en el kernel, recurrencia y sesgos, además de un Dropout de 0.5. Finalmente, al igual que en la RNN simple, se utilizó una capa densa con Softmax para la clasificación en tres clases y el optimizador Adam con tasa de aprendizaje de 0.0001.

3.3. Configuración del entrenamiento

La configuración del entrenamiento se ajustó para gestionar el desbalance de clases presente en el conjunto de datos, donde la categoría negativa supera en número a las categorías

positiva y neutral. Para equilibrar el impacto del número de clases durante el aprendizaje, se calcularon pesos de clase (class weights) antes de iniciar el proceso. Utilizando la heurística 'balanced' de la librería scikit-learn, se asignó un valor numérico a cada clase inversamente proporcional a su frecuencia de aparición. De esta manera, la función de pérdida penaliza en mayor medida los errores de clasificación cometidos en las clases minoritarias (neutral y positivo) en comparación con la clase mayoritaria, forzando al algoritmo a considerar todas las categorías por igual.

Las funciones de pérdida utilizadas fueron entropía cruzada categórica dispersa (sparse_categorical_crossentropy) y entropía cruzada categórica (categorical_crossentropy). La diferencia principal entre ambas radica en el formato de las etiquetas: la versión sparse trabaja con etiquetas representadas como enteros, mientras que la versión estándar requiere etiquetas codificadas en one-hot.

En el modelo MLP se empleó sparse_categorical_crossentropy porque las etiquetas de la primera entrega estaban representadas directamente como índices, lo que hacía innecesaria la codificación one-hot y permitía un entrenamiento más eficiente. En cambio, en los modelos RNN simple y LSTM se utilizó categorical_crossentropy, ya que en la segunda entrega las etiquetas ya se encontraban codificadas en formato one-hot, por lo que correspondía usar la variante estándar.

En todos los modelos se aplicó el optimizador Adam, ajustando manualmente una tasa de aprendizaje, se empezó con una tasa de aprendizaje de 0.001, sin embargo, los modelos presentaron sobreajuste, especialmente los de RNN y LSTM, por ello se bajó la tasa hasta 0.0001 donde hubo un equilibrio en el entrenamiento y de esta forma se asegura una convergencia estable y evitar fluctuaciones en la curva de pérdida.

El proceso de entrenamiento se estructuró con un tamaño de lote (batch size) de 32 muestras. El número de épocas fue diferente para cada modelo, el modelo MLP se entrenó con 100 épocas, mientras que los modelos de RNN simple y LSTM se entrenaron con 40 y 30 épocas respectivamente, controlado mediante un mecanismo de parada temprana (Early Stopping). Este callback se configuró para monitorear la pérdida en el conjunto de validación (val_loss) con una paciencia de 5 épocas en todos los modelos. Si el rendimiento en validación no mejoraba dentro de este margen, el entrenamiento se detenía automáticamente.

Adicionalmente, para los modelos RNN simple y LSTM se activó el parámetro restore_best_weights=True para asegurar que el modelo final conservara los pesos de la época con menor error de validación, descartando los cambios realizados en las épocas posteriores donde el rendimiento comenzaba a degradarse.

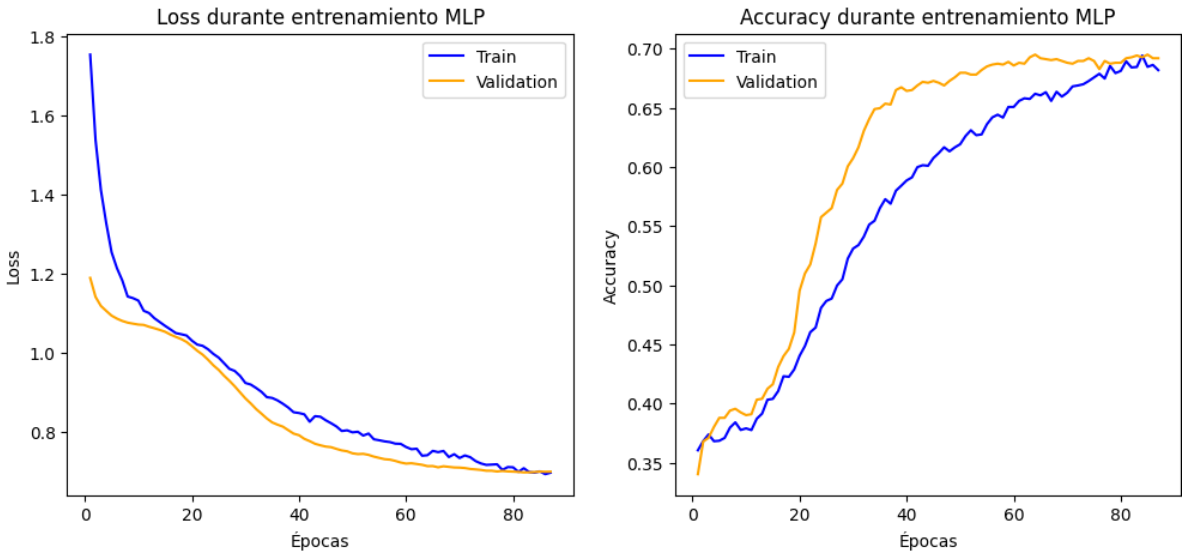
En cuanto a la regularización, se aplicaron estrategias combinadas para limitar la complejidad de los modelos. En la capa de entrada de las redes secuenciales se utilizó SpatialDropout1D, el cual anula dimensiones completas de los vectores de características para reducir la dependencia de términos específicos. En las capas ocultas se aplicó Dropout estándar con tasas de 0.3 para el modelo RNN simple y una tasa de 0.2 para LSTM. Específicamente para la arquitectura LSTM, se añadió regularización L2 con factores de 0.001 aplicados a los pesos del kernel, la recurrencia y los sesgos. Esta restricción sobre la magnitud de los pesos tiene como objetivo suavizar las curvas de aprendizaje y prevenir que el modelo memorice el ruido del conjunto de entrenamiento.

3.4. Análisis de resultados

Para evaluar los modelos, se realizó una medición de desempeño utilizando el conjunto de prueba, el cual corresponde al 15% de los datos totales y representa ejemplos que la red neuronal nunca vio durante su entrenamiento. Debido a la naturaleza desbalanceada del problema, donde la mayoría de las opiniones son negativas, no es suficiente con observar únicamente la exactitud global (Accuracy). Un modelo podría tener una exactitud alta simplemente prediciendo siempre la clase mayoritaria, pero fallaría en detectar los comentarios positivos o neutros, que son igual de importantes. Por esta razón, el análisis se centra en observar el comportamiento de las curvas de aprendizaje y en desglosar las métricas de Precisión, Sensibilidad (Recall) y F1-Score para cada modelo.

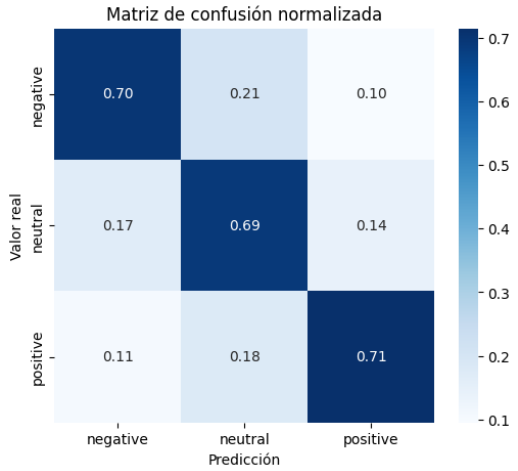
3.4.1. Evaluación del Perceptrón Multicapa (MLP)

El primer modelo evaluado corresponde al enfoque MLP utilizando representaciones TF-IDF y una red neuronal densa. A continuación, se presentan las curvas de aprendizaje obtenidas durante el entrenamiento:



Como se observa en la figura anterior, el modelo MLP muestra un proceso de aprendizaje estable. La curva de pérdida de validación (línea naranja) desciende rápidamente durante las primeras épocas y logra estabilizarse cerca de la pérdida de entrenamiento, lo cual indica que el modelo no sufrió de un sobreajuste severo. La exactitud en validación crece de manera sostenida hasta alcanzar un punto estable alrededor de 0.70 y en la época 80 aproximadamente, momento en el cual el Early Stopping detiene el proceso para conservar los mejores pesos. Esto sugiere que la reducción de dimensionalidad con SVD ayudó a simplificar la entrada lo suficiente para que la red densa encontrara patrones generales sin memorizar ruido excesivo.

En cuanto al desempeño numérico en el conjunto de prueba, el modelo alcanzó una exactitud global del 70%. Sin embargo, el análisis más detallado de las métricas de evaluación se obtuvo del reporte de clasificación y de la matriz de confusión.

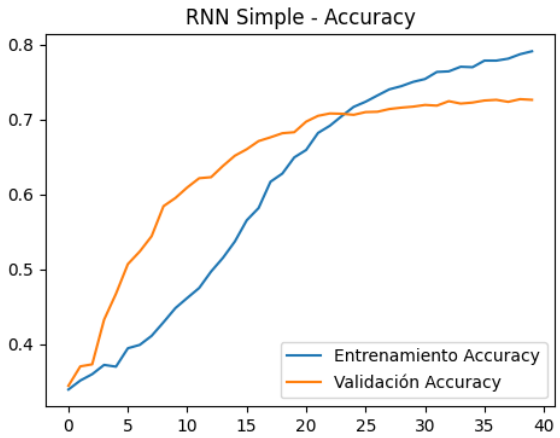


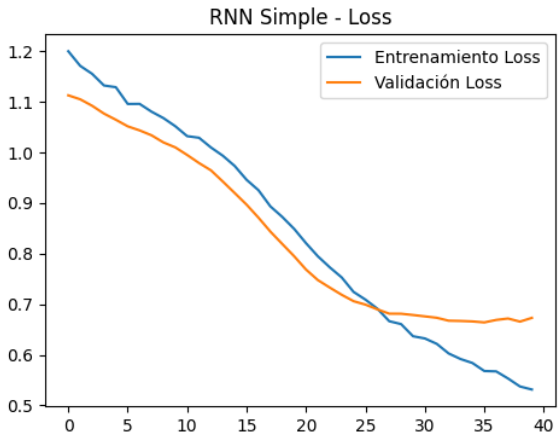
Clase	Precisión	Recall	F1-score	Support
Negativa	0.89	0.70	0.78	1835
Neutral	0.48	0.69	0.57	620
Positiva	0.56	0.71	0.63	473
Accuracy	0.70			

Este reporte de clasificación indica que, a pesar de las limitaciones con respecto a otros modelos, este modelo MLP logró un desempeño notablemente equilibrado. Gracias a la implementación de los pesos de clase (class weights), el sistema no ignoró las categorías minoritarias. Esto se evidencia en la columna de Recall (Sensibilidad), donde se obtuvo un 0.69 para la clase neutral y un 0.71 para la positiva. Esto significa que el modelo es capaz de detectar correctamente el 71% de los comentarios positivos reales. La matriz de confusión confirma visualmente este hallazgo, mostrando una diagonal principal con valores de 0.70, 0.69 y 0.71. Si bien la precisión en la clase neutral es baja (0.48), lo que indica que el modelo tiende a confundir algunos tweets negativos como neutros, el hecho de que logre identificar la mayoría de los casos positivos demuestra que la estrategia de vectorización TF-IDF con ponderación de clases fue una muy buena implementación, considerando que antes de aplicar esta técnica se obtuvo métricas muy por debajo de las que se obtuvo después de aplicarla.

3.4.2. Evaluación de la Red Neuronal Recurrente Simple (RNN)

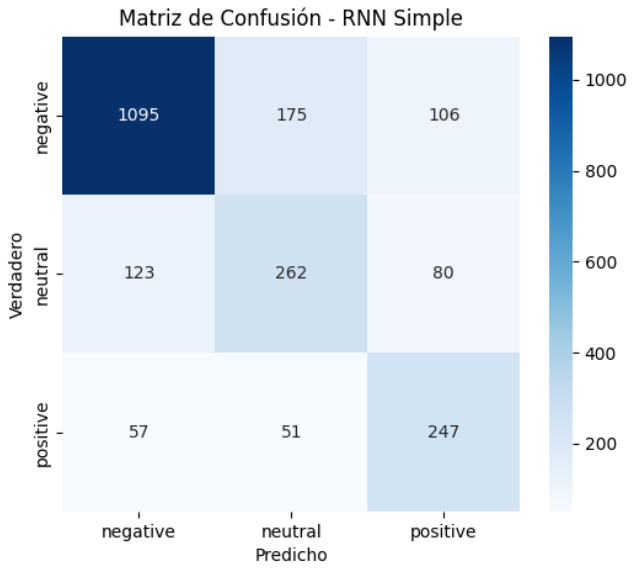
En la segunda etapa del proyecto, se implementaron modelos secuenciales capaces de procesar el orden de las palabras. El primer lugar se implementó la RNN Simple, esta red procesa el texto token por token. A continuación, se muestra el comportamiento de su entrenamiento.





Al analizar las gráficas de entrenamiento, se identifica un fenómeno distinto al del modelo MLP. La curva de exactitud en validación (línea naranja) asciende rápidamente hasta alcanzar un pico cercano al 73% aproximadamente en la época 40, pero posteriormente se estanca. Simultáneamente, la curva de pérdida de validación muestra un descenso inicial brusco, alcanzando su mínimo óptimo alrededor de la época 25, luego comienza a mantener el mismo valor e incluso aumentando ligeramente mientras la pérdida de entrenamiento sigue bajando. Este comportamiento de divergencia se interpreta como un temprano sobreajuste (overfitting). Sin embargo, gracias a la configuración del Early Stopping con restauración de los mejores pesos, el entrenamiento se detuvo, evitando que la red degradara su capacidad de generalización y conservando el modelo en su punto de máximo rendimiento

A continuación. se presenta la matriz de confusión junto con el reporte de clasificación con las métricas como Accuracy general, exactitud, precisión, etc. Con estas medidas se puede concluir sobre qué tan bueno se comportó el modelo al momento de hacer las predicciones sobre cada clase.



Clase	Precisión	Recall	F1-score	Support
Negativa	0.86	0.80	0.83	1376
Neutral	0.54	0.56	0.55	465
Positiva	0.57	0.70	0.63	355
Accuracy	0.73			

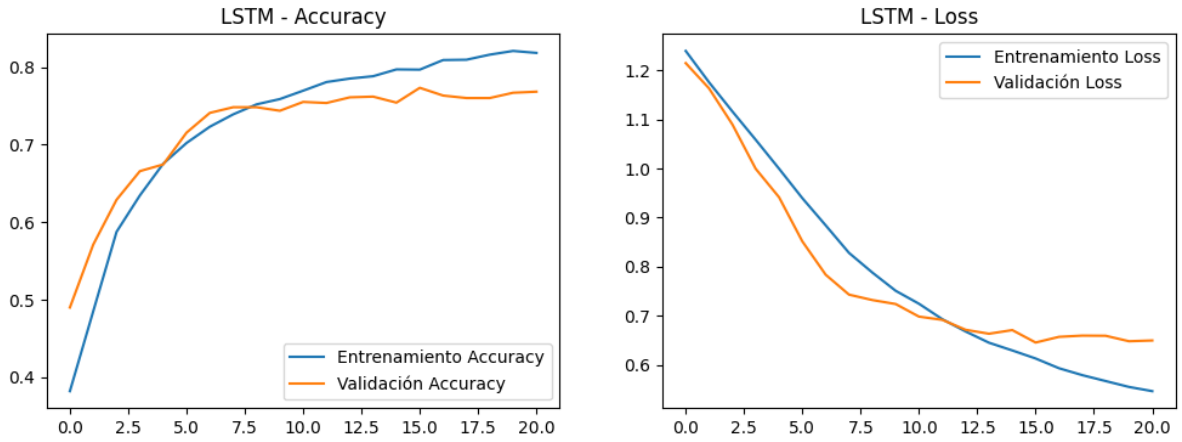
El modelo RNN Simple alcanzó una exactitud global (Accuracy) del 73%, superando ligeramente al MLP. No obstante, al mirar detalladamente el reporte de clasificación y la matriz de confusión, se observa un comportamiento desigual entre clases. La mejora en la exactitud global se debe principalmente a un aumento en la detección de la clase mayoritaria "negativa”, cuyo Recall subió a 0.80 (comparado con el 0.70 del MLP).

Sin embargo, esta arquitectura mostró debilidades significativas en la clase "neutral". El Recall para los neutros descendió a 0.56, lo que indica que la red confunde casi la mitad de los comentarios neutrales, clasificándolos erróneamente como negativos (123 casos) o positivos (80 casos), como se evidencia en la segunda fila de la matriz de confusión. Esto sugiere que, debido a la falta de memoria a largo plazo característica de las RNN simples (problema de desvanecimiento del gradiente), el modelo tiene dificultades para retener el contexto sutil necesario para distinguir la neutralidad en frases donde no hay palabras emocionales fuertes, tendiendo a sesgarse hacia la clase dominante "negativa" con más

facilidad que el modelo MLP. Por otro lado, la detección de positivos se mantuvo bien con un Recall de 0.70, demostrando que las expresiones de satisfacción son lo suficientemente distintivas para ser captadas por una recurrencia simple.

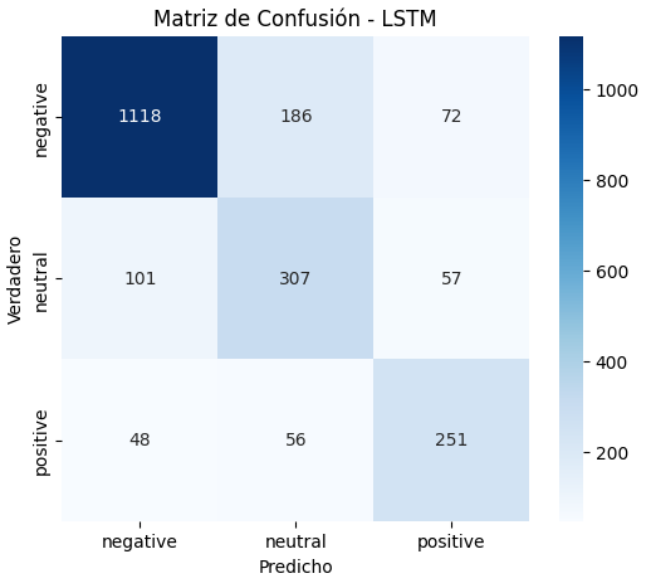
3.4.3. Evaluación de la red neuronal LSTM

Finalmente, se evaluó la arquitectura de la red LSTM. Este modelo fue diseñado específicamente para superar las limitaciones de memoria de la RNN Simple, incorporando celdas que deciden qué información guardar y cuál olvidar a lo largo de la secuencia del tweet. A continuación, se presentan sus curvas de entrenamiento:



Al observar el comportamiento gráfico de la LSTM, se nota una diferencia en la velocidad de aprendizaje comparada con los modelos anteriores. La red converge más rápido, pues a diferencia de las anteriores, esta converge alrededor de la época 20, gracias a la implementación de Early Stopping, de esta forma se evita el sobreajuste al entrenar durante muchas épocas. La curva de pérdida de validación (línea naranja) alcanzó su punto más bajo alrededor de la época 15 y luego comienza a subir suavemente, mientras que la exactitud se mantiene estable por encima del 75%. Este patrón indica que la red es muy buena y tiene muy buena capacidad para aprenderse el dataset, por lo que las técnicas de regularización L2 y el Early Stopping fueron vitales para detener el entrenamiento, evitando que el modelo empezara a memorizar ruido (sobreajuste) y garantizando que se guardaran los pesos del momento óptimo.

Los resultados del reporte de clasificación de las métricas junto con la matriz de confusión se presentan a continuación:



Clase	Precisión	Recall	F1-score	Support
Negativa	0.88	0.81	0.85	1376
Neutral	0.56	0.66	0.61	465
Positiva	0.66	0.71	0.68	355
Accuracy	0.76			

El modelo LSTM alcanzó la exactitud global más alta, con un 0.76 (76%). Además, este modelo recuperó las clases que la RNN Simple había perdido. Al analizar el reporte, se observa que el Recall de la clase "neutral" subió a 0.66, una mejora de 10 puntos porcentuales respecto a la RNN simple. Esto significa que la memoria de la LSTM le permitió entender contextos ambiguos o carentes de palabras emocionales fuertes, clasificando correctamente 307 tweets neutros (como se ve en la matriz), frente a los 262 de la RNN.

De igual forma, la clase "positiva" mantuvo un rendimiento sólido con un Recall de 0.71 y una precisión de 0.66. La matriz de confusión muestra una diagonal principal mucho más equilibrada que en los casos anteriores. En conclusión, aunque el MLP fue un buen punto de partida y la RNN Simple mejoró la detección de clases negativas, la LSTM demostró ser la arquitectura más completa, logrando el mejor balance entre precisión y sensibilidad gracias a su capacidad para procesar la secuencia completa del texto y gestionar el desbalance de clases de manera efectiva.

4. Conclusiones

Al realizar las pruebas con los diferentes modelos, se pudo ver la diferencia entre usar una red neuronal clásica y las redes que manejan secuencias. El modelo MLP dio resultados buenos, pero se notó que le faltaba capacidad para entender el sentido completo de las frases, ya que solo se basaba en el conteo de palabras. En cambio, al pasar a los modelos RNN y LSTM, el uso de Embeddings ayudó a que la red pudiera interpretar mejor el orden de las palabras, lo cual es necesario para saber si un tweet es realmente una queja o solo un comentario neutro.

De los tres modelos evaluados, la red LSTM fue la que mejor funcionó. Aunque la RNN Simple mejoró un poco la precisión general comparada con el MLP, tuvo problemas para distinguir los comentarios neutros y tendía a clasificarlos como negativos, probablemente porque olvida rápido el inicio de la frase. La LSTM solucionó esto gracias a su memoria interna, logrando identificar mucho mejor los tweets positivos y neutros. Esto demuestra que, para este tipo de textos, donde el contexto importa mucho, es mejor usar una red que pueda recordar la información por más tiempo en lugar de una red simple.

Por último, se comprobó que la configuración del entrenamiento es igual de importante que elegir el modelo. Como había muchos más tweets negativos que de los otros tipos, fue obligatorio usar los pesos de clase (class weights). Sin esto, los modelos aprendían a predecir siempre "negativo" y fallaban en todo lo demás. También fue clave usar la regularización y el Early Stopping, especialmente con la LSTM, porque al principio el modelo memorizaba los datos muy rápido y dejaba de aprender. Con estos ajustes se logró que las gráficas de entrenamiento se estabilizaran y que el modelo final pudiera hacer predicciones correctas en datos que no había visto antes.

5. Bibliografía

DataCamp. (2024). *Predicciones bursátiles con LSTM en Python*. Obtenido de Data Science Tutorials: https://www.datacamp.com/es/tutorial/lstm-python-stock-market

IBM. (10 de 06 de 2025). *Redes perceptrón multicapa*. Obtenido de IBM SPSS Statistics: https://www.ibm.com/docs/en/spss-statistics/31.0.0?topic=networks-multilayer-perceptron

kaggle. (s.f.). *Twitter US Airline Sentiment*. Obtenido de kaggle datasets: https://www.kaggle.com/datasets/crowdflower/twitter-airline-sentiment

MathWorks. (2024). *LSTM — Long Short-Term Memory layer*. Obtenido de MATLAB Deep Learning: https://la.mathworks.com/help/deeplearning/ref/dlarray.lstm.html

MongoDB. (s.f.). *Recurrent Neural Networks (RNN): What they are and how they work*. Obtenido de MongoDB Resources: https://www.mongodb.com/resources/basics/artificial-intelligence/recurrent-neural-networks